Standard Change for Pegasus v2

By Mário Pires Verified 8 min 32 Add a reaction

This Change is part of the Change Catalog RE Platform

Summary

Pegasus v2 has several components that can be delivered independently. However, new features might need changes to more than one component to be fully functional.

This document aims at showing the entire SDLC for Pegasus v2, so that it can be used as the standard change catalog article for any change in Pegasus v2.

The changes that allowed to be done with this catalog:

- Spacelift Pegasus v2 stacks
- Pegasus v2 infrastructure
- Pegasus v2 configs
- Pegasus v2 lambdas
- Pegasus v2 Athena bot

When we need to change several components for a single change, the expected order for the deployment of those changes is as presented above.

What is NOT allowed (and thus require a dedicated catalog & RFC):

- Direct changes in Pegasus v2 databases
- Any other change not “as code”

General Information

Change Type	Standard
Approved Normal RFC	RFC-2824: Promote to Standard - Pegasus v2 changes COMPLETED
Change Risk	Low/Medium Risk Matrix Calculation **Note:** any High risk will merit a dedicated Normal RFC
Downtime Involved	No **Note:** if there is a downtime, we’ll create a dedicated Normal RFC
Rollback Available	Yes - we’ll be always able to revert to the previous Pegasus version
Implementation Team	Release Engineering Alpha Team
Component(s)/Asset(s)	Pegasus v2
Impact Description	*Dependent on the change - should be part of the description of the SRFC*

SRFC Content

SRFC Description

Since the agreement is that we can have multiple changes in multiple repos in a single SRFC, the description of that SRFC should contain the following:

- Risk assessment and mitigation strategy
  - If existing code/functionality is changed, how are we guaranteeing it's not going to fail?
  - If new code/functionality is added, how are we guaranteeing it's working?
  - Are new tests going to be created? What kind of tests (unit, system, e2e)?
- Possible impact in Pegasus v1 components

If any additional change is needed that was not planned, we must update the description.

Example: RFC-2852: Pegasus v2 - support Rollback for Self-hosted DUs COMPLETED

SRFC Comments

After the SRFC is approved and during the implementation, we must add comments (or a single comment) with all opened PRs related with the change. Once we start merging, we must add the link to the Spacelift run that applied those changes to Production.

Spacelift Pegasus v2 Stacks

Changes done in repo https://github.com/OutSystems/spacelift-admin-stack Connect your Github account
in files `admin-stack/reng-alpha-aws-pegasus-*`

These files contain the definition of the Spacelift Stacks for Pegasus infrastructure.

> ⚠ Any deletion of resources in the stack definition should have its own RFC for review & approval.

We have a Lab account where we do all our changes validations. Prod account, as the name suggests, contains the Production infrastructure.

- Lab and Prod have their own stacks - there should always be one stack for Lab and one Stack for prod for the same terraform project.
- Prod stacks have `github_action_deploy = false` to block changes from PR directly, so changes need to be merged to main to then be applied via Spacelift.

Procedure to do a change:

- A SRFC is created describing the change (and all other changes to other components)
- PR is opened with the change to the Lab stack
- PR is "promoted" in Spacelift to the Lab stack, to apply the changes to the AWS Lab account, so that tests can validate the change
- Change is validated (probably requires changes in other components)
- Change is done in Prod stack (same PR)
- Code review verifies the same change was done in Prod stack
- Evidences that Lab is working with the change are collected and added to the PR description
- PR is approved by at least 1 Code Reviewer
- PR is merged to main
- Change is applied to Prod in Spacelift
- SRFC is closed (after all involved components changes are also merged/applied)

PR Automatic Validations (and requirements):

- Spacelift plan (aka "proposed")

> ℹ If a change only aims at changing the Lab stack (e.g. for a PoC), no SRFC is required.

Changes done in repository https://github.com/OutSystems/pegasus-infra Connect your GitHub account
in folders aws-core and/or aws-sh.

These folders have the infrastructure-as-code definition of a Pegasus v2 infrastructure.

AWS-Core contains the core Pegasus v2 infrastructure - infra that can be used by more than 1 Core Lambda.

Both Lab and Prod stacks are using these folders to get the terraform scripts to apply. The Stacks defined in Standard Change for Pegasus v2 | 3. Spacelift Pegasus v2 Stacks ensure the resources are created in the corresponding accounts.

Procedure to do a change:

- A SRFC is created describing the change (and all other changes to other components)
- PR is opened with the change
- PR is "promoted" in Spacelift to Lab, to apply the changes to the AWS Lab account, so that tests can validate the change
- Change is validated (probably requires changes in other components)
- Evidences that Lab is working with the change are collected and added to the PR description
- PR is reviewed and approved by at least 1 Code Reviewer
- PR is merged to main
- Change is applied to Prod in Spacelift
- SRFC is closed (after all involved components changes are also merged/applied)

PR Automatic Validations (and requirements):

- Terraform formatter
- Terraform linter
- Spacelift plan (aka "proposed")

Pegasus v2 Configs Changes

Changes done in repository https://github.com/OutSystems/pegasus-configs Connect your Github account ,
in the context of a Pegasus v2 change.

This repo has 3 parts:

- configs (yaml files)
- parser and linters (python package)
- generated infra (generated terraform files)

The **configs**, and especially the Lifecycle configs, affect how and what Pegasus will do to apply a change process for ODC assets.

The **parser** is a python package to load the configs into memory as python classes. This package is versioned and should be consumed as a normal python package in the Core lambdas project.

The **generated infra** is the terraform configs that are generated from the configs, more notably, the AWS state machines that represent the processes defined in the Lifecycles.

We have configs for Lab separate from configs for Prod. While this allows changing Lab without changing Prod, it also means that we need to ensure the configs for Prod mirror the configs for Lab after validations. However, they could diverge (e.g. Lab can have more lifecycles, or lifecycles with different processes), for testability reasons. One thing is for certain:

> ⚠ Lab should always have a Lifecycle with the same processes as Prod, so we can validate that those processes work.

- PR is created describing the change (and all other changes to other components)
- PR is opened with the change (Prod changes can be included already)
- (if applicable) PR is "promoted" in Spacelift to the Lab stack, to apply the changes to the AWS Lab account, so that tests can validate the change
- (if applicable) If PR has labels `push_configs_to_s3` and `run_in_lab`, the Lab configs will be pushed to S3 (Lab account)
- Change is validated (probably requires changes in other components)
- Evidences that Lab is working with the change are collected and added to the PR description
- PR is reviewed and approved by at least 1 Code Reviewer
- PR is merged to main
- (if applicable) Change is applied to Prod in Spacelift
- (if applicable) A new version of the python package is created to be consumed by other Pegasus components
- Prod configs are pushed to S3 (Prod account)
- SRFC is closed (after all involved components changes are also merged/applied)

PR Automatic Validations (and requirements):

- Unit tests for parser (and linters)
- Custom linters for configs, ensuring the configs are valid
- Backward compatibility / breaking change detection - ensuring previous version of parser can load the current configs
- Detection of unsynchronized configs changes vs corresponding generated terraform files
- SonarCloud code analysis
- Snyk analyses
- Spacelift plan (aka "proposed")

> ⚠ A new version of the Parser is not immediately used by the Lambdas. The Lambdas python project needs to be updated to use the new version of the parser, following the process described in Standard Change for Pegasus v2 | 6. Pegasus v2 Lambdas Changes.

> ℹ If a change aims at only changing the Lab configs (e.g. for a PoC), no SRFC is required.

Pegasus v2 Lambdas Changes

Changes done in these repos:

- https://github.com/OutSystems/pegasus-lambdas-core Connect your Github account
- https://github.com/OutSystems/pegasus-lambdas-commercial Connect your Github account
- https://github.com/OutSystems/pegasus-lambdas-sh Connect your Github account
- https://github.com/OutSystems/pegasus-extensions Connect your Github account

These repos contain the infrastructure and lambdas implementation for Pegasus Core logic, for Commercial processes logic and for Self-Hosted processes logic.

Although these are different repos, we follow the same development processes when doing changes.

The infrastructure is defined in folder `app/infra`. Both Lab and Prod stacks are using this folder to get the terraform scripts to apply.

All lambdas in a repo will have the same code base from `app/src`. This is a python project that will be installed into a docker container image. The lambdas will then use that image as their container image (using the image SHA). Each lambda uses a different function as entrypoint.

that image as source.

Procedure to do a change:

- A SRFC is created describing the change (and all other changes to other components)
- PR is opened with the change
- If there are infra changes, PR must be "promoted" in Spacelift to the Lab stack, to apply the changes to the AWS Lab account, so that tests can validate the change
- If there are python code changes, the PR must have the label `push_to_ecr`, so that the image that is built in that PR is pushed to the **Lab's ECR** for testing; a Spacelift run is then automatically triggered to recreate the Lambdas with the new image SHA
- Change is validated (probably requires changes in other components)
- Evidences that Lab is working with the change are collected and added to the PR description
- PR is reviewed and approved by at least 1 Code Reviewer
- PR is merged to main
- Main GHW will build the new lambdas image and push it to Prod's ECR, and a new Spacelift run is automatically created; change is applied to Prod in Spacelift
- SRFC is closed (after all involved components changes are also merged/applied)

PR Automatic Validations (and requirements):

- Unit tests for Lambdas implementations
- Python code linters
- Docker build (if label is present)
- SonarCloud code analysis
- Snyk analyses (code & image)
- Spacelift plan (aka "proposed")

Pegasus v2 Athena Bot

Changes done in repo https://github.com/OutSystems/pegasus-bot Connect your Github account and file https://github.com/OutSystems/pegasus-infra/blob/main/az-configs/container_app_config.tf Connect your Github account
in the context of a Pegasus v2 change.

These repos contains the code for our Athena Bot, and the corresponding Azure App where the bot is deployed.

We have 2 bots deployed: `Athena-Lab` and `Athena`. Any change we do is validated first in `Athena-Lab`.

The `Athena-Lab` app uses the latest image of the bot available in EdenStage, while `Athena` uses a fixed version defined in file https://github.com/OutSystems/pegasus-infra/blob/main/az-configs/container_app_config.tf Connect your Github account available in EdenCore.

Procedure to do a change:

- A SRFC is created describing the change (and all other changes to other components)
- PR is opened with the change
- PR will build & push the new docker image to EdenStage, becoming the latest version for `Athena-Lab`
- Change is validated (probably requires changes in other components) by restarting the `Athena-Lab` App in Azure's UI
- Evidences that Lab is working with the change are collected and added to the PR description
- PR is reviewed and approved by at least 1 Code Reviewer
- PR is merged to main
- Main GHW will build the new image and push it to EdenCore

us-infra/blob/main/az-configs/container_app_config.tf Connect your Github account

- PR is reviewed and approved by at least 1 Code Reviewer
- PR is merged to main
- Spacelift is automatically triggered to update the `Athena` App in Azure
- SRFC is closed (after all involved components changes are also merged/applied)

PR Automatic Validations (and requirements):

- Unit tests for Athena implementation
- Python code linters
- Docker build
- SonarCloud code analysis
- Snyk analyses (code & image)
- Spacelift plan (aka "proposed")

Manual Testing

We have the following manual testing strategies:

- Lambdas Component tests - validating that the lambda entrypoints work from our machine, using our credentials;
- Lambdas System tests - validating that the aws lambdas created in Lab work, but only focusing on the components from that repo (e.g. Core logic only);
- E2E tests - validating that the entire Pegasus v2 system works, from the 1st event to the last event of all affected product lines.

The mandatory tests to run to ensure any change is ok are the E2E tests. Any PR should show evidences of these tests passing.

For now these tests are manual, but we would like to make them automatic at some point.

About Security

Main Branch Protections

All main branches have the standard protection ruleset:

- Require signed commits
- Require a pull request before merging
  - 1 required approval
  - Require review from Code Owners
  - Require conversation resolution before merging
  - Dismiss stale pull request approvals when new commits are pushed
- Require status checks to pass (sonarcloud + unit tests + docker build)
  - Require branches to be up to date before merging
  - Do not require status checks on creation
- Block force pushes

Github AWS permissions

Github Actions use federation to connect to the AWS accounts. PR actions are only allowed to change the Lab account; Main actions are allowed to change both Lab and Prod accounts.

Those actions are further limited by AWS Policies that restrict the operations the actions can take (e.g. in configs repo, the action can only push the configs to S3).

How Reverts Work

Since all changes are “as code”, we can revert any merge to main we do, followed by a Spacelift run (if applicable).

Related content Send feedback

[Normal] RE Change - Routine Pegasus Infra Change
Nelson Fonseca

[Normal] RE Change - Routine Pegasus Extensions Change
Felipe Coimbra Fonseca

[Normal] RE Change - Routine Pegasus Config Change
Vitor Rolo

[Normal] RE Change - Routine Pegasus Pipeline Change
Leandro Galrinho

[Normal] SDLC Change - Routine Pegasus Flux Extension Sidecar Change
Edson Junior Rodrigues de Morais

[Normal] SDLC Change - Routine Pegasus Bot Change
Nelson Fonseca

Add a comment NEW-ENTRY-CATALOG-RD-TEA... PEGASUS Add a reaction